

Inheritance in Python - Complete Notes

Inheritance in Python - Complete Notes

1. Definition:

Inheritance in Python allows one class (child/subclass) to inherit attributes and methods from another class (parent/superclass).

It promotes code reusability and creates a relationship between classes.

Example:

class Parent:

```
def display(self):  
    print("Parent class")
```

class Child(Parent):

```
def show(self):  
    print("Child class")
```

obj = Child()

obj.display()

obj.show()

Output:

Parent class

Child class

Real-world Example:

A 'Car' class inheriting from a generic 'Vehicle' class.

2. Why Use Inheritance?

- To reuse code instead of rewriting.

Inheritance in Python - Complete Notes

- To extend or modify the behavior of a parent class.
- To represent real-world hierarchies.

3. Syntax:

class Parent:

parent methods and attributes

class Child(Parent):

additional or overridden methods and attributes

4. Types of Inheritance in Python

A. Single Inheritance:

A child class inherits from one parent class.

Example:

class Vehicle:

def start(self):
 print("Vehicle started")

class Car(Vehicle):

def drive(self):
 print("Car is driving")

obj = Car()

obj.start()

obj.drive()

Real-world Example:

'Car' inheriting from 'Vehicle'.

Inheritance in Python - Complete Notes

B. Multilevel Inheritance:

A class is derived from another derived class.

Example:

```
class Grandparent:
```

```
    def feature1(self):  
        print("Feature 1")
```

```
class Parent(Grandparent):
```

```
    def feature2(self):  
        print("Feature 2")
```

```
class Child(Parent):
```

```
    def feature3(self):  
        print("Feature 3")
```

```
obj = Child()
```

```
obj.feature1()
```

```
obj.feature2()
```

```
obj.feature3()
```

Real-world Example:

'Manager' -> 'Employee' -> 'Person' hierarchy.

C. Multiple Inheritance:

A child class inherits from multiple parent classes.

Example:

```
class Father:
```

```
    def skill1(self):
```

Inheritance in Python - Complete Notes

```
print("Driving")
```

```
class Mother:
```

```
    def skill2(self):
```

```
        print("Cooking")
```

```
class Child(Father, Mother):
```

```
    def skill3(self):
```

```
        print("Dancing")
```

```
obj = Child()
```

```
obj.skill1()
```

```
obj.skill2()
```

```
obj.skill3()
```

Real-world Example:

A 'Smartphone' inheriting from 'Camera' and 'Phone'.

D. Hierarchical Inheritance:

Multiple child classes inherit from the same parent class.

Example:

```
class Parent:
```

```
    def show(self):
```

```
        print("Parent method")
```

```
class Child1(Parent):
```

```
    def feature1(self):
```

```
        print("Feature 1")
```

```
class Child2(Parent):
```

Inheritance in Python - Complete Notes

```
def feature2(self):  
    print("Feature 2")
```

```
obj1 = Child1()  
obj2 = Child2()  
obj1.show()  
obj2.show()
```

Real-world Example:

Different account types ('SavingsAccount', 'CurrentAccount') inheriting from 'BankAccount'.

E. Hybrid Inheritance:

Combination of two or more inheritance types.

Example:

```
class A:  
    def displayA(self):  
        print("Class A")
```

```
class B(A):  
    def displayB(self):  
        print("Class B")
```

```
class C(A):  
    def displayC(self):  
        print("Class C")
```

```
class D(B, C):  
    def displayD(self):  
        print("Class D")
```

Inheritance in Python - Complete Notes

```
obj = D()
obj.displayA()
obj.displayB()
obj.displayC()
obj.displayD()
```

Real-world Example:

A 'Smart Device' inheriting from 'Phone' and 'Computer'.

5. Method Overriding:

When a child class defines a method with the same name as in the parent class.

Example:

```
class Parent:
```

```
    def greet(self):
        print("Hello from Parent")
```

```
class Child(Parent):
```

```
    def greet(self):
        print("Hello from Child")
```

```
obj = Child()
```

```
obj.greet()
```

Output:

Hello from Child

6. super() Keyword:

Used to call methods of the parent class inside a child class.

Example:

Inheritance in Python - Complete Notes

```
class Parent:
    def show(self):
        print("Parent method")
```

```
class Child(Parent):
    def show(self):
        super().show()
        print("Child method")
```

```
obj = Child()
obj.show()
```

Output:

Parent method

Child method

7. Method Resolution Order (MRO):

Determines the order in which Python looks for methods in case of multiple inheritance.

Example:

```
print(Child.__mro__)
```

8. Real-world Example:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def show_details(self):
        print(f"Employee: {self.name}, Salary: {self.salary}")
```

Inheritance in Python - Complete Notes

```
class Manager(Employee):  
    def __init__(self, name, salary, department):  
        super().__init__(name, salary)  
        self.department = department  
  
    def show_details(self):  
        super().show_details()  
        print(f'Department: {self.department}')  
  
m1 = Manager("Sai Kiran", 75000, "Data Analytics")  
m1.show_details()
```

Output:

Employee: Sai Kiran, Salary: 75000

Department: Data Analytics

9. Key Points:

- Inheritance promotes code reuse.
- super() calls parent methods/constructors.
- MRO defines the method search order.
- Types: Single, Multiple, Multilevel, Hierarchical, Hybrid.